Project 3: DQN for Atari Breakout

Mina Fahmy | DS551/CS551 | Worcester Polytechnic Institute | Fall 2026

1. Objective and environment

I implemented a Deep Q-Network (DQN) to play Breakout. The assignment target is an average reward above 40 over 100 evaluation episodes, with five lives per episode. The latest reported evaluation achieved **274.52**; an earlier evaluation achieved **280.55**.

The project uses the instructor-provided BreakoutNoFrameskip-v4 environment and Atari wrappers. Each observation contains four stacked 84 x 84 grayscale frames. Inputs are converted to channel-first format and pixel values are divided by 255 inside the network. The instructor-provided environment and testing files are kept unchanged.

2. Network and learning method

The network has three convolutional layers: 32 filters with an 8 x 8 kernel and stride 4; 64 filters with a 4 x 4 kernel and stride 2; and 64 filters with a 3 x 3 kernel and stride 1. ReLU follows each layer. The 3,136 flattened features feed a 512-unit ReLU layer and a linear output layer with four action values. PyTorch default weight initialization is used.

The agent uses epsilon-greedy exploration, uniform experience replay, and a separate target network. Adam optimizes Huber loss, with gradient norms clipped to 10. The learning target is reward + gamma x (1 - terminated) x max Q_target(next state).

3. Training settings

Setting	Value	Setting	Value
Training steps	3,000,000	Replay capacity	50,000
Batch size	32	Learning rate	0.0001
Discount factor	0.99	Replay warm-up	10,000 steps
Update frequency	Every 4 steps	Target synchronization	Every 10,000 steps
Training epsilon	1.0 to 0.1 over 1M steps	Test epsilon	0.01
Agent seed	595	Evaluation seed	11037

4. Experiment performed

The documented experiment trained this baseline configuration for 3,000,000 agent steps on an NVIDIA A30 GPU on WPI Turing. Training took approximately 6,423 seconds (1 hour and 47 minutes), and the final checkpoint was saved as dqn.pth. The job requested four CPU cores and 16 GB of host memory. This report does not claim architecture comparisons or hyperparameter sweeps that were not performed.

Learning Curve and Results

5. Training learning curve

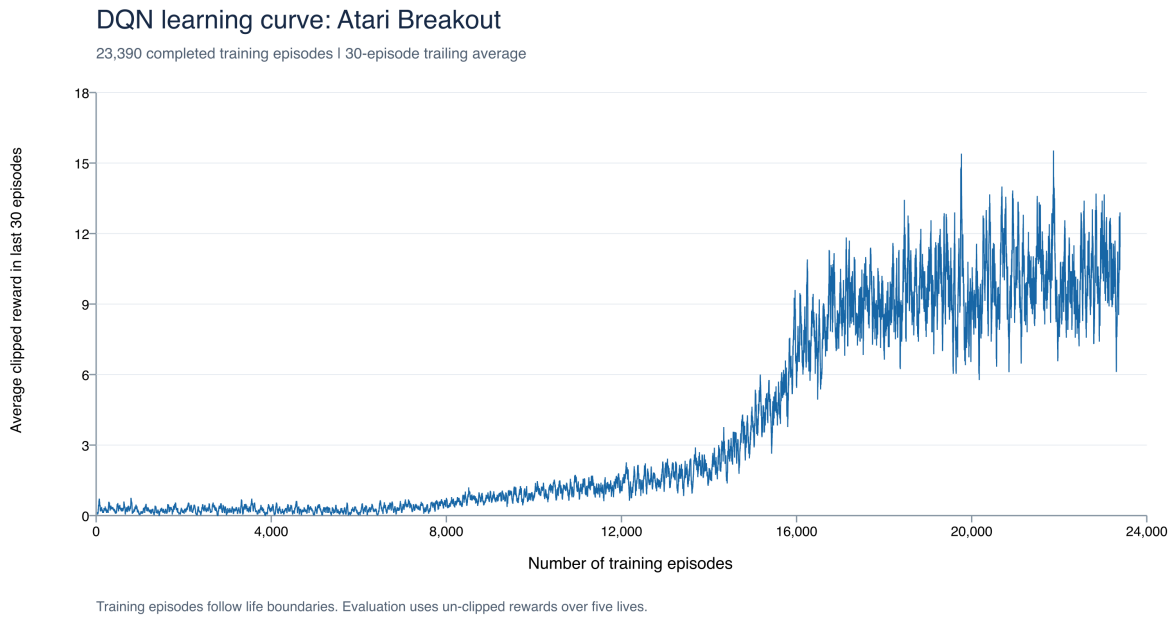


Figure 1. Average clipped reward over the last 30 training episodes versus episode number. The plot uses all full 30-episode windows, beginning at episode 30.

The CSV contains **23,390 completed training episodes**. Recalculated rolling means match every logged mean30 value. Its final row ends at step 2,999,624; the training log confirms the final checkpoint at step 3,000,000.

Training measure	Result
Mean reward in first 30 episodes	0.13
Mean reward in final 30 episodes	11.43
Highest 30-episode mean	15.53 at episode 21,868
Mean reward in first / final 1,000 episodes	0.28 / 10.31

Rewards increase substantially during training, but fluctuate late in the run. The final mean remains below the peak, so convergence is not established. Training uses clipped rewards and life-level episodes; its values differ from un-clipped, five-life evaluation rewards.

6. Evaluation and conclusion

Evaluation	Mean reward	Episodes / lives
Earlier reported run	280.55	100 / 5 per episode
Latest reported run	274.52	100 / 5 per episode

Both evaluations exceed the target of 40. The latest evaluation loop took about 1,060 seconds. The outputs do not establish checkpoint identity across runs, so the score difference is not attributed to additional training. No independent-seed comparisons or ablations were performed.

Sources: Supplied training.csv and reported training/evaluation logs. Assignment: WPI DS551/CS551 Project 3 instructions.